DATA STRUCTURES

26CS3IPDST

Semester:	**3**		
Course Title:	**Data Structures**		
Course Code:	**26CS3IPDST**	**Total Contact Hours:**	**42**
L-T-P:	**3-0-1**	**Total Credits:**	**4**

Unit No.	Topics	Hours
1	**Introduction**: Data and Information, Data Structures, Classification of Data Structures, Data Structures versus File Organization, Operations on Data Structures, Introduction to Structures. **Stacks**: Introduction to Stacks, Stack Operations, Array Representations of Stacks, Stack Applications : Infix to Postfix Conversion, Postfix Expression Evaluation, Function Call Tracing, Recursion.	8
2	**Queues**: Introduction to Queues, Queue Operations, Array Representations of Queues, Circular Queues, Priority Queues, Double Ended Queues. **Pointers and Dynamic Memory Allocation.** **Linked Lists**: Introduction, Singly Linked List Implementation, Insertion, Deletion and Searching Operations on Singly Linked Lists.	9
3	**Linked Lists**: Circular Linked Lists: Insertion, Deletion and Searching Operations for Circular Linked Lists. Doubly Linked Lists: Insertion, Deletion and Searching Operations for Doubly Linked Lists. **Applications of Linked Lists** : Stack and Queue Implementation, Manipulation of Polynomials (addition), Representing Sparse Matrices..	9

4	**Trees**: Introduction, Tree Representation, Properties of Trees. **Binary Trees**: Binary Tree Representation, Binary Tree Properties, Binary Tree Traversals, Binary Tree Implementation, Binary Search Trees, Binary Search Tree Operations and Implementation.	8
5	**Graphs**: Introduction, Trees versus Graphs, Graph Traversals: Breadth-First Search (BFS) - Examples and Implementation, Depth-First Search (DFS) - Examples and Implementation. **Hashing Techniques**: Hash Table Representation, Hash functions, Collision Resolution By Separate Chaining, Collision Resolution By Open Addressing-Linear Probing, Quadratic Probing, Double Hashing, Cuckoo Hashing, Hopscotch Hashing.	8

Prescribed Text Books:

Sl. No.	Book Title	Authors	Edition	Publisher	Year
1	Fundamentals of Data Structures in C	Horowitz Sahni, Anderson Freed	2nd	Universities Press	2008
2	Data Structures using C	Aaron M.Tenenbaum, Yedidyah Langsam, Moshe J. Augenstein	5th	Pearson Education	2007

Reference Text Books:

Sl. No.	Book Title	Authors	Edition	Publisher	Year
1	Data Structures using C	Reema Thareja	2nd	Oxford University Press	2014
2	An Introduction to Data Structure with Applications	Jean - Paul Tremblay, Paul Sorenson	2nd	Tata MacGraw Hill	2007
3	Cracking the Coding Interview	Gayle Laakmann		CareerCup	2010
4	A Simplified Approach to Data Structures	Lalit Goyal, Vishal Goyal, Pawan Kumar	1st	SPD	2014

E-Books:

Sl. No.	Book Title	Authors	Edition	Publisher	Year	URL
1	Data Structures using C	Reema Thareja	2nd	Oxford University Press	2014	https://sagarmittal1.github.io/3-DSA/assets/books/Data%20structures%20using%20C%20-%20Reema%20Thareja.pdf
2	Cracking the Coding Interview	Gayle Laakmann		CareerCup	2010	http://www.r-5.org/files/books/computers/overviews/cs-tools/Gayle_Laakmann-Cracking_the_Coding_Interview-EN.pdf

MOOC Courses:

Sl. No.	Course name	Course Offered By	Year	URL
1	Data Structures	Coursera	2022	https://www.coursera.org/learn/data-structures
2	Data Structures and Algorithms	NPTEL	2022	https://nptel.ac.in/courses/106102064/

Course Outcomes (COs):

CO1 (WK3)	Apply the concepts of linear and non-linear data structures to represent data for a given problem.
CO2 (WK3)	Analyse the given problem to determine appropriate data structure and provide solutions.
CO3 (WK5)	Develop solutions for a given problem using appropriate linear and non-linear data structures.
CO4 (WK5, WK6)	Conduct practical experiments for implementing different data structures.

UNIT - 1

Introduction: Data and Information, Data Structure, Classification of Data Structures, Data structure versus File Organization, Operations on Data Structure. Introduction to Structures.

Stacks - Introduction to Stacks, Stack Operations, Array representations of stacks, Stack applications: Infix to postfix conversion, Postfix expression evaluation, Function call tracing, Recursion.

• Data and Information

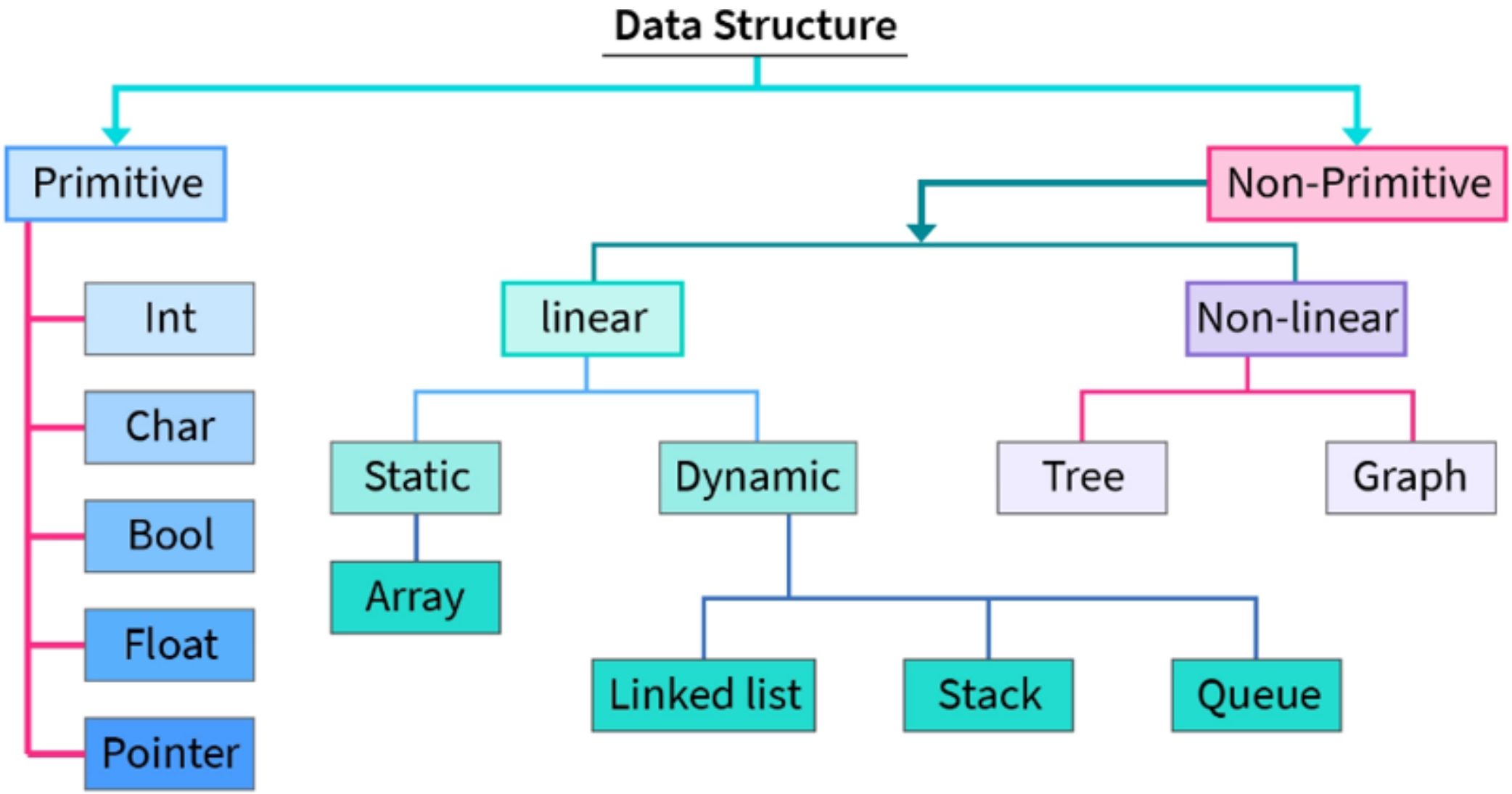
Data consists of raw, unorganized facts, figures, symbols, or observations, such as numbers, words, or images, that lack meaning on their own.

Information is data that has been processed, organized, and given context, making it meaningful, comprehensible, and useful for making decisions or understanding a situation.

• Data Structure

- ✓ A data structure is a data organization, management, and storage format that enables efficient access and modification.
- ✓ Data structures serve as the basis for abstract data types (ADT). The ADT defines the logical form of the data type. The data structure implements the physical form of the data type.
- ✓ Some examples of Data Structures are arrays, Linked List, Stack, Queue, etc. Data Structures are widely used in almost every aspect of Computer Science i.e. **Operating System, Compiler Design, Artificial intelligence, Graphics and many more.**

- **Classification of Data Structures**



- **Data Structures are mainly classified into two categories:**
- **Primitive Data structure**
  - ✓ They store the data of only one type i.e. **built-in data types.**
  - ✓ Data types like integer, float, character, and Booleans come in this category.
- **Non- Primitive Data structure**
  - ✓ They can store the data of more than one type i.e. derived data types.
  - ✓ Data types like arrays, linked lists, trees, etc. come in this category.
  - ✓ Non-Primitive Data Structures are data structures derived from Primitive Data Structures.
- They further divided into **two categories:**
  - ✓ **Linear Data Structures:** The data in this data structure is arranged in a sequence, one after the other i.e. each element appears to be connected linearly. Based on the memory allocation, the Linear Data Structures are again classified into two types: **Static Data Structures and Dynamic Data Structures**
  - ✓ **Non-Linear Data Structures:** The data in this data structure are not arranged in a sequence. They form a hierarchy i.e. the data elements have multiple ways to connect to other elements. **e.g. tree and graph**

Advantage of Data structures

- Efficiency
- Reusability
- Abstraction
- Easy Data Processing
- Develop Algorithms
- Reusability of Data
- Provide Built-in Functions
- Data Structure Selection for Optimal Performance
- Saves Programmer’s Time
- Practical Applications
- Increased Data Security

• Data structure versus File organization

Data Structures:

Focus:

Organizing data within main memory (RAM) for efficient processing and manipulation by programs.

Purpose:

To store and retrieve data efficiently, optimize algorithmic performance, and manage relationships between data elements.

Examples:

Arrays, linked lists, trees (e.g., binary search trees, B-trees), hash tables, stacks, queues, graphs.

Scope:

Primarily concerned with the logical arrangement and access methods for data during program execution.

File Organization:

Focus:

Organizing data on secondary storage devices (e.g., hard drives, SSDs) for persistent storage and retrieval.

Purpose:

To manage the physical storage of data, minimize disk access times, and facilitate efficient long-term storage and retrieval of files.

Examples:

Sequential file organization, indexed sequential file organization, direct/hash file organization, B-tree file organization.

Scope:

Concerned with how data is physically laid out on disk, how records are located, and how files are managed by the operating system's file system.

Feature	Data Structures	File Organization
Location	Main Memory (RAM)	Secondary Storage (Disk)
Purpose	Efficient processing and manipulation	Efficient persistent storage and retrieval
Scope	Program execution, logical organization	Physical storage, file system management
Longevity	Temporary (during program execution)	Permanent (persistent storage)

Operations on Data Structure

- Traversing
- Insertion
- Deletion
- Searching
- Sorting
- Merging
- Updating/Modification
- Access

Introduction to Structures

- Structures are user-defined data types that allow the grouping of variables of different data types under a single name. This provides a way to represent complex entities with multiple attributes, such as a student record containing a name (string), roll number (integer), and marks (float).
- Key aspects of C structures:

  Declaration: Structures are declared using the **`struct`** keyword, followed by a tag name (optional) and a block containing the member variables.

```
struct Student
{
char name[50];
int roll_number;
float marks;
};
```

```
struct Student s1;// Declares a
                  structure variable named s1

s1.roll_number = 101;

printf("Student Name: %s\n", s1.name);
```

```
struct Student s2 = {"Alice", 102, 85.5};  //Initializer list

struct Student s3 = { .name = "Bob", .roll_number = 103, .marks = 90.0 };
                                                    // Designated initializers
```

`typedef` with Structures: The **`typedef`** keyword can be used to create an alias for a structure type, simplifying its usage by avoiding the need to repeatedly use the **`struct`** keyword.

```
typedef struct Student
{
char name[50];
int roll_number;
float marks;
} StudentInfo;   // StudentInfo is now an alias for struct Student

StudentInfo s4;  // Declare using the alias
```

- WAP for the below given scenario:

A university wants to automate their admission process. Students are admitted based on the marks scored in a qualifying exam. A student is identified by student id, age and marks in qualifying exam.

Data are valid, if:

  - Age is greater than 20
  - Marks is between 0 and 100 (both inclusive)

A student qualifies for admission,

  if Age and marks are valid, and Marks is 65 or more

- Write a program to represent the students seeking admission in the university.

```
#define SIZE 100
  struct student
  {
  int id;
  int age;
  int marks;
  };
```

```
int main()
{
 struct student s[SIZE];
 int no_of_students,i;

 printf("Please enter no. of students")
 scanf("%d",&no_of_students);
 for(i=0;i<no_of_students;i++)
 {
   accept_input(s[i]);
 }

for(i=0;i<no_of_students;i++)
 {
   display_output(s[i]);
 }

   return 0;
}
```

```
void accept_input( struct student s1)
{
 printf("enter the id");
 scanf("%d",&s1.id);
 printf("enter the age");
 scanf("%d",&s1.age);

 printf("enter the marks");
 scanf("%d",&s1.marks);

}
```

Write logic

```
void display_output( struct student s1)
{
 printf("sudent details are:\n");
 printf("ID - %d\t" s1.id);
 printf("ID - %d\t" s1.age);
 printf("ID - %d\n" s1.marks);
}
```

Stack: Representation of stacks in C

- A stack is an Abstract Data Type (ADT) and linear data structure that follows the **Last in, First Out(LIFO)** principle of data insertion and deletion. It means that the element that is inserted last will be the first one to be removed and the element that is inserted first will be removed at last.,

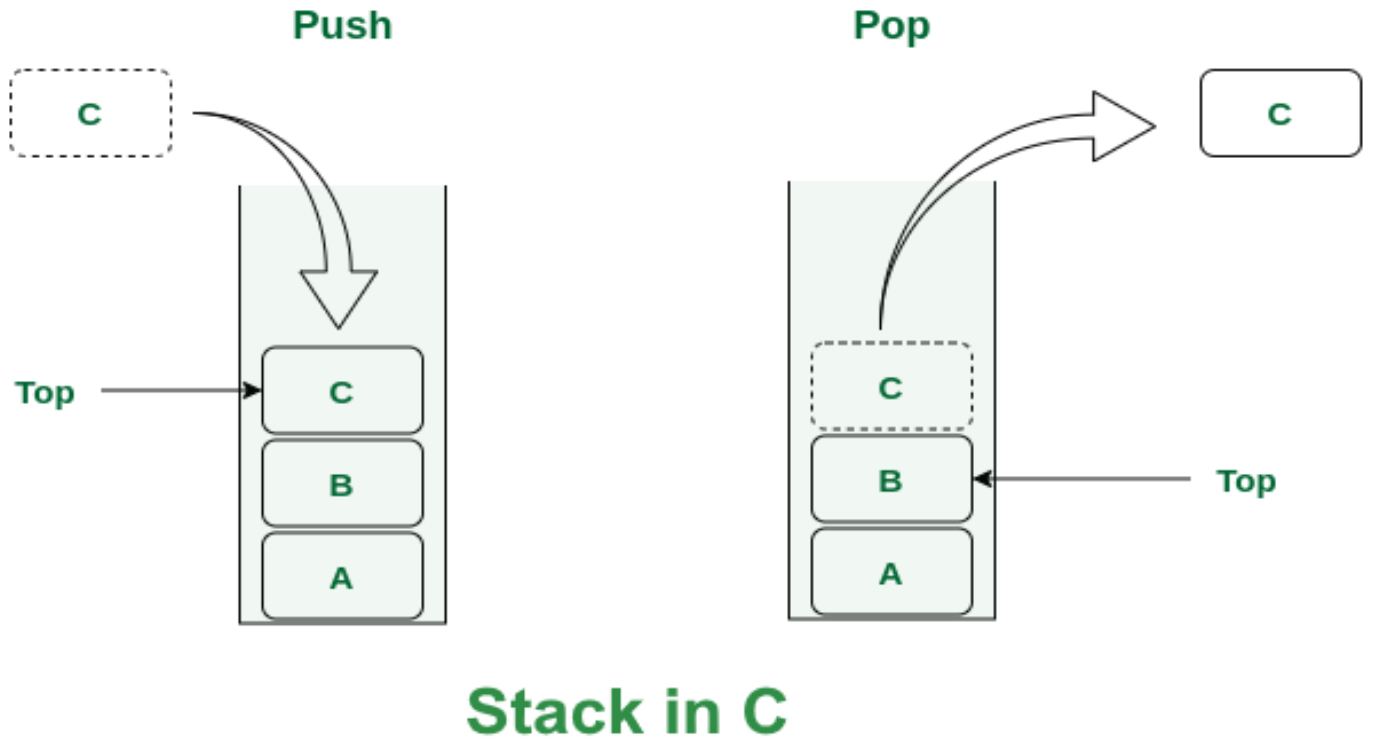





Stack Operations

Basic operations:

- push() – Pushing (storing) an element on the stack.
- pop() – Removing (accessing) an element from the stack.



Operation	Description
Push	Inserts an element at the top of the stack.
Pop	Removes the top most element of the stack.
Peek	Returns the topmost element of the stack.
IsFull	Returns true is the stack is full otherwise returns false.
IsEmpty	Returns true is the stack is empty otherwise returns false.

Applications of Stack in C

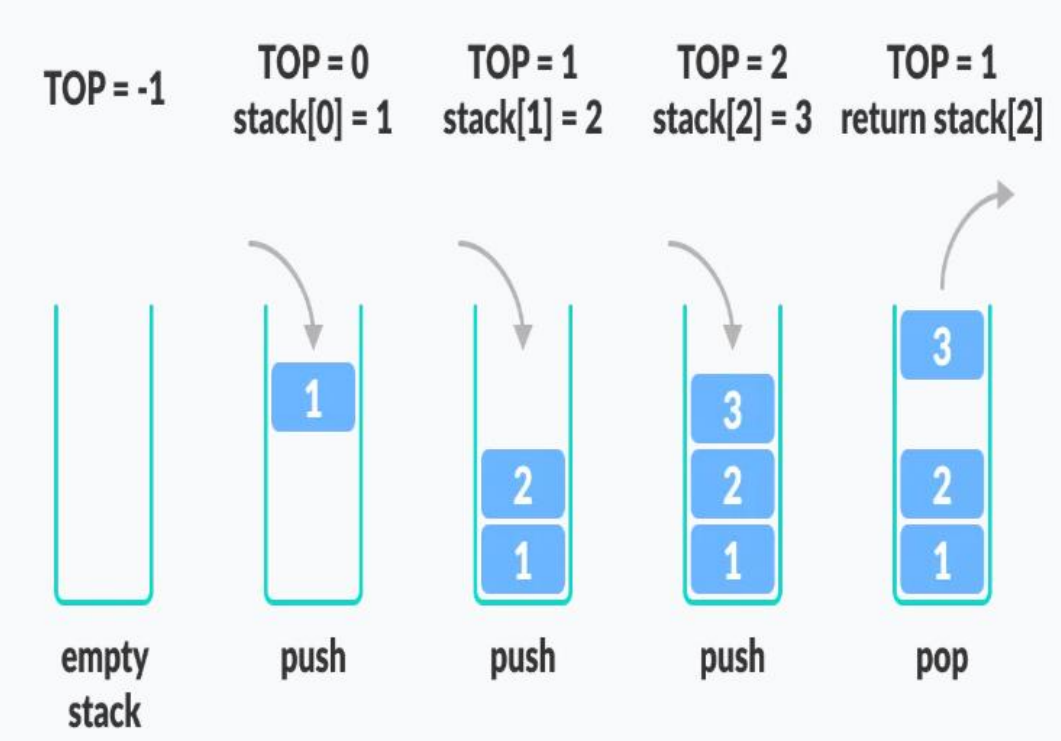
Stack is widely used for Following are some common applications of Stack:

- Stacks are commonly used to **evaluate postfix expressions.** It is also used in infix to postfix conversion.
- It is used in **Function calls/recursion** where a different stack is allocated for every recursive call.
- It is used in **browsers to provide the backward and forward functionality**.
- It is also used in **text editor, image editors** to provide the undo and redo functionality.
- It is used in **various algorithms** in computer science
- For **Backtracking.**

Implementation of Stack

- Implementation of a stack using
  - **Arrays**
  - **Linked List (In Unit 3)**

- **Initialization:** The `top` variable is initialized `to -1` to indicate an empty stack.
- **Operations:**
  - **Push:** Increments `top` and adds the new element at `items[top].` Checks for stack overflow (if `top` reaches `MAX_SIZE - 1`).
  - **Pop:** Decrements `top` after retrieving the element at `items[top].` Checks for stack underflow (if `top` is -1).
  - **Peek:** Returns the element at `items[top]` without removing it.
  - **isEmpty:** Checks if `top is -1.`
  - **isFull:** Checks if `top` is `MAX_SIZE - 1.`



```
Step 1: IF TOP = MAX-1
            PRINT "OVERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
```

Figure 7.7 Algorithm to insert an element in a stack

```
Step 1: IF TOP = NULL
            PRINT "STACK IS EMPTY"
            Goto Step 3
Step 2: RETURN STACK[TOP]
Step 3: END
```

Figure 7.11 Algorithm for Peek operation

```
Step 1: IF TOP = NULL
            PRINT "UNDERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

Figure 7.10 Algorithm to delete an element from a stack

```c
#include<stdio.h>
#include<conio.h>

#define SIZE 10

void push(int);
void pop();
void display();

int stack[SIZE], top = -1;

void main()
{
   int value, choice;
   clrscr();
   while(1){
      printf("\n\n***** MENU *****\n");
      printf("1. Push\n2. Pop\n3. Display\n4. Exit");
      printf("\nEnter your choice: ");
      scanf("%d",&choice);
      switch(choice){
         case 1: printf("Enter the value to be insert: ");
                 scanf("%d",&value);
                 push(value);
                 break;
         case 2: pop();
                 break;
         case 3: display();
                 break;
         case 4: exit(0);
         default: printf("\nWrong selection!!! Try again!!!");
      }
   }
}
```

```
void push(int value){
   if(top == SIZE-1)
      printf("\nStack is Full!!! Insertion is not possible!!!");
   else{
      top++;
      stack[top] = value;
      printf("\nInsertion success!!!");
   }
}
  void pop(){
     if(top == -1)
        printf("\nStack is Empty!!! Deletion is not possible!!!");
     else{
        printf("\nDeleted : %d", stack[top]);
        top--;
     }
  }
```

```
void display(){
   if(top == -1)
      printf("\nStack is Empty!!!");
   else{
      int i;
      printf("\nStack elements are:\n");
      for(i=top; i>=0; i--)
         printf("%d\n",stack[i]);
   }
}
```

Example: Reverse a string

```c
/*function definition of pushChar*/
void pushChar(char item)
{
    /*check for full*/
    if(isFull())
    {
        printf("\nStack is FULL !!!\n");
        return;
    }

    /*increase top and push item in stack*/
    top=top+1;
    stack_string[top]=item;
}
```

```c
/*function definition of popChar*/
 char popChar()
{
    /*check for empty*/
    if(isEmpty())
    {
        printf("\nStack is EMPTY!!!\n");
        return 0;
    }

    /*pop item and decrease top*/
    item = stack_string[top];
    top=top-1;
    return item;
}
```

```c
/*stack variables*/
int top=-1;
int item;
```

```c
/*string declaration*/
char stack_string[MAX];
```

```c
int main()
{
    char str[MAX];

    int i;

    printf("Input a string: ");
    scanf("%[^\n]s",str); /*read string with spaces*/
    /*gets(str);-can be used to read string with spaces*/

    for(i=0;i<strlen(str);i++)
        pushChar(str[i]);

    for(i=0;i<strlen(str);i++)
        str[i]=popChar();

    printf("Reversed String is: %s\n",str);

    return 0;
}
```

```c
/*function definition of isFull*/
int isFull()
{
    if(top==MAX-1)
        return 1;
    else
        return 0;
}
```

```c
/*function definition of isEmpty*/
int isEmpty()
{
    if(top==-1)
        return 1;
    else
        return 0;
}
```

OR

```
#include<stdio.h>
#include<string.h>
#define size 20
int top = -1;
char stack[size];

char push(char ch)
{
if(top==(size-1))
printf("Stack is Overflow\n");
else
stack[++top]=ch;
}

char pop()
{
if(top==-1)
printf("Stack is Underflow\n");
else
return stack[top--];
}

int main()
{
char str[20];
int i;
printf("Enter the string : \n" );
gets(str);
for(i=0;i<strlen(str);i++)
{
push(str[i]);
}
for(i=0;i<strlen(str);i++)
{
   str[i]=pop();
}
printf("Reversed string is : ");
puts(str);

}
```

OUTPUT:

```
Enter the string:    Reverse of String
Reversed string is: gnirtS fo esreveR
```

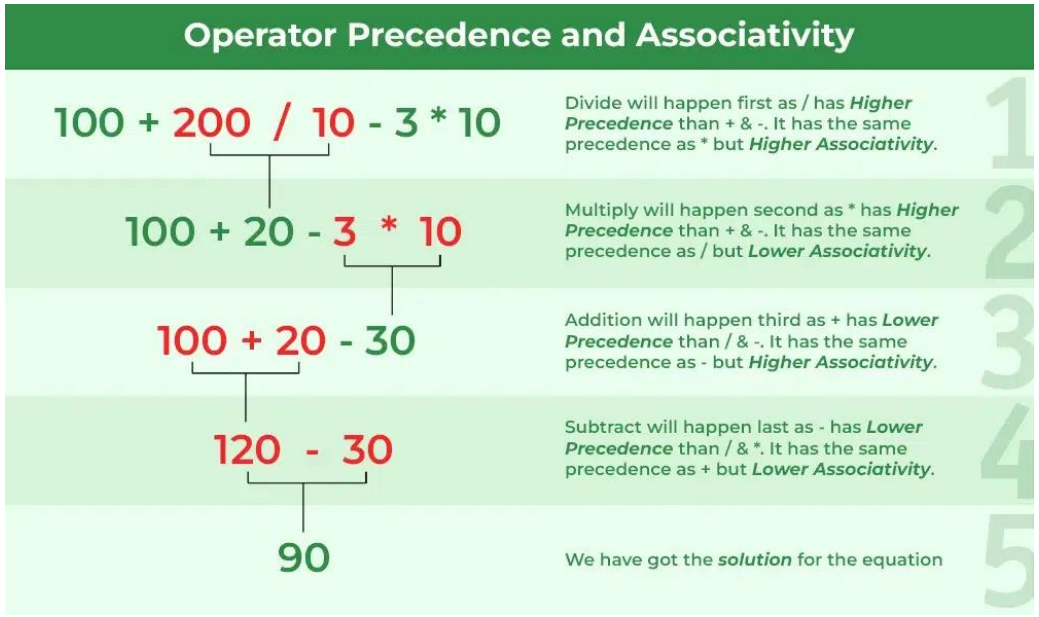
Infix, Prefix and Postfix Expressions

- Infix, Postfix and Prefix notations are three different but equivalent ways of writing expressions.

Infix Expression	Prefix Expression	Postfix Expression
A + B * C + D	+ + A * B C D	A B C * + D +
(A + B) * (C + D)	* + A B + C D	A B + C D + *
A * B + C * D	+ * A B * C D	A B * C D * +
A + B + C + D	+ + + A B C D	A B + C + D +

Advantage of Postfix Expression over Infix Expression

- An infix expression is difficult for the machine to know and keep track of precedence of operators. On the other hand, a postfix expression itself determines the precedence of operators (as the placement of operators in a postfix expression depends upon its precedence).
- Therefore, for the machine it is easier to carry out a postfix expression evaluation than an infix expression.




```
int result4 = 10 - (4 / 2);


int x = 5;
int result = ++x * 2;

int val = 10;
val += 5 * 2;
```

int exp = 100 + 200 / 10 - 3 * 10;

int x = 5 - 17* 6

```
int result1 = 5 + 3 * 2;

int result2 = 10 - 4 / 2;

int result3 = (5 + 3) * 2;
```

Operator Precedence

Operators have different levels of precedence, which determine the order of evaluation:

^ (Exponentiation) Highest precedence

*, / (Multiplication, Division) Higher precedence

+, -, % (Addition, Subtraction, modulus) Lowest precedence

Example 7.1 Convert the following infix expressions into postfix expressions.

Solution

```
(a) (A-B) * (C+D)
     [AB-] * [CD+]
     AB-CD+*
(b) (A + B) / (C + D) - (D * E)
     [AB+] / [CD+] - [DE*]
     [AB+CD+/] - [DE*]
     AB+CD+/DE*-
```

Infix to Postfix Conversion Algorithm

The algorithm for converting an infix expression to a postfix expression using a stack is:

Read the expression from left to right.

Step 1. If an operand (A-Z, 0-9) is encountered, add it directly to the output.

Step 2. If an operator is encountered:

i. Pop operators from the stack to the output if they have higher or equal precedence.

ii. Push the current operator onto the stack.

Step 3. If an opening parenthesis (is found, push it onto the stack.

Step 4. If a closing parenthesis) is found, pop operators from the stack to the output until an opening parenthesis is encountered.

Step 5. After scanning the entire expression, pop any remaining operators from the stack to the output.

Example: Convert Infix to Postfix

Infix expression: A + B * C

Step	Scanned Character	Stack	Postfix Expression
1	A	---	A
2	+	+	A
3	B	+	A B
4	*	+ *	A B
5	C	+ *	A B C
6	-	---	A B C * +

Final Postfix Expression: A B C * +

Example 1:

Convert ((A – (B + C)) * D) ↑ (E + F) infix expression to postfix form:

SYMBOL	POSTFIX STRING	STACK	REMARKS
(		(	
(		((	
A	A	((	
-	A	((-	
(	A	((- (	
B	A B	((- (	
+	A B	((- (+	
C	A B C	((- (+	
)	A B C +	((-	
)	A B C + -	(	
*	A B C + -	(*	
D	A B C + - D	(*	
)	A B C + - D *		
↑	A B C + - D *	↑	
(	A B C + - D *	↑ (	
E	A B C + - D * E	↑ (	
+	A B C + - D * E	↑ (+	
F	A B C + - D * E F	↑ (+	
)	A B C + - D * E F +	↑	
End of string	A B C + - D * E F + ↑	The input is now empty. Pop the output symbols from the stack until it is empty.	

Example (Infix to Postfix)

- Infix: A*B+C*D-E

  AB*+C*D-E

  AB*+CD*-E

  AB*CD*+-E

- Postfix: AB*CD*+E-

Infix Expression: **A+ (B*C-(D/E^F)*G)*H**, where **^** is an exponential operator.

Symbol	Scanned	STACK	Postfix Expression	Description
1.		(		Start
2.	A	(	A	
3.	+	(+	A	
4.	(	(+(	A	
5.	B	(+(	AB	
6.	*	(+(*	AB	
7.	C	(+(*	ABC	
8.	-	(+(-	ABC*	'*' is at higher precedence than '-'
9.	(	(+(-(	ABC*	
10.	D	(+(-(	ABC*D	
11.	/	(+(-(/	ABC*D	
12.	E	(+(-(/	ABC*DE	
13.	^	(+(-(/^	ABC*DE	
14.	F	(+(-(/^	ABC*DEF	
15.	)	(+(-	ABC*DEF^/	Pop from top on Stack , that's why '^' Come first
16.	*	(+(-*	ABC*DEF^/	
17.	G	(+(-*	ABC*DEF^/G	
18.	)	(+	ABC*DEF^/G*-	Pop from top on Stack , that's why '^' Come first
19.	*	(+*	ABC*DEF^/G*-	
20.	H	(+*	ABC*DEF^/G*-H	
21.	)	Empty	ABC*DEF^/G*-H*+	END

(a) A – (B / C + (D % E * F) / G)* H

Infix Character Scanned	Stack	Postfix Expression
	(	
A	(	A
–	(–	A
(	(– (	A
B	(– (	A B
/	(– (/	A B
C	(– (/	A B C
+	(– (+	A B C /
(	(– (+ (	A B C /
D	(– (+ (	A B C / D
%	(– (+ (%	A B C / D
E	(– (+ (%	A B C / D E
*	(– (+ (% *	A B C / D E
F	(– (+ (% *	A B C / D E F
)	(– (+	A B C / D E F * %
/	(– (+ /	A B C / D E F * %
G	(– (+ /	A B C / D E F * % G
)	(–	A B C / D E F * % G / +
*	(– *	A B C / D E F * % G / +
H	(– *	A B C / D E F * % G / + H
)		A B C / D E F * % G / + H * –

ABC/DE%F*G/+H*-

```c
#include <stdio.h>
#include <ctype.h>

#define SIZE 50

char stack[SIZE];
int top=-1;       /* Global declarations */


    /* Main Program */
  void main()
  {
      char infix[50],postfix[50],ch,elem;
      int i=0,k=0;
      printf("Enter Infix Expression : ");
      scanf("%s",infix);
      push('#');
      while( (ch=infix[i++]) != '\0')
      {
          if( ch == '(') push(ch);
          else
              if(isalnum(ch)) postfix[k++]=ch;
              else
                  if( ch == ')')
                  {
                      while( stack[top] != '(')
                          postfix[k++]=pop();
                      elem=pop(); /* Remove ( */
                  }
                  else
                  {       /* Operator */
                      while( pr(stack[top]) >= pr(ch) )
                          postfix[k++]=pop();
                      push(ch);
                  }
      }
      while( stack[top] != '#')     /* Pop from stack till empty */
          postfix[k++]=pop();
      postfix[k]='\0';          /* Make postfix as valid string */
      printf("\nPostfix Expression =  %s\n",postfix);
  }
```

```
push(char elem)
{                       /* Function for PUSH operation */
    stack[++top]=elem;
}

char pop()
{                      /* Function for POP operation */
    return(stack[top--]);
}
```

```
int pr(char symbol)
{                  /* Function for precedence */

        if(symbol == '^')/* exponent operator, highest precedence*/
        {
                return(3);
        }
        else if(symbol == '*' || symbol == '/')
        {
                return(2);
        }
        else if(symbol == '+' || symbol == '-')          /* lowest precedence */
        {
                return(1);
        }
        else
        {
                return(0);
        }
}
```

Evaluation of the postfix expression

Now that we have converted infix to postfix, let's see how to evaluate it:

1. Read the postfix expression from left to right.
2. If an operand is found, push it to the stack.
3. If an operator is found, pop the last two operands from the stack, apply the operation, and push the result back.
4. Continue until the expression is fully processed. The final value in the stack is the result.

```c
#include <stdio.h>
#include <ctype.h>

int stack[40];
int top = -1;

void push(int x)
{
   stack[++top] = x;
}

int pop()
{
   return stack[top--];
}

int main()
{
   char exp[20];
   char ch;
   int n1,n2,n3,num, i=0;
   printf("Enter the expression: ");
   scanf("%s", exp);

   while((ch=exp[i++]) != '\0')
   {
      if(isdigit(ch))
      {
         num = ch - 48;
         push(num);
      }
```

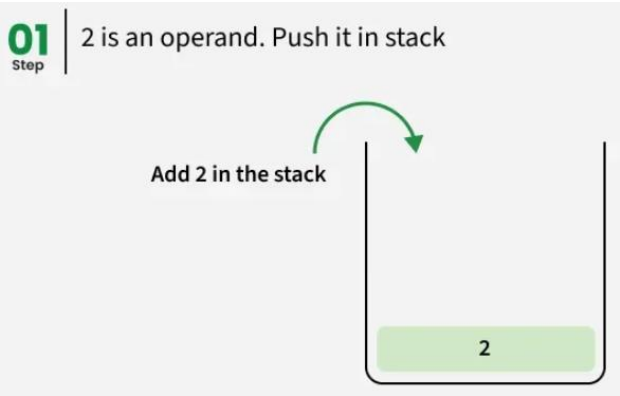
```
else
        {
          n1 = pop();
          n2 = pop();
         switch(ch )
         {
          case '+':
          {
             n3 = n1 + n2;
             break;
          }
          case '-':
          {
             n3 = n2 - n1;
             break;
          }
          case '*':
          {
             n3 = n1 * n2;
             break;
          }
          case '/':
          {
             n3 = n2 / n1;
             break;
          }
          }
          push(n3);
       }
    }
printf("\nThe result of expression %s %d\n\n",  exp,  pop());
return 0; }
```

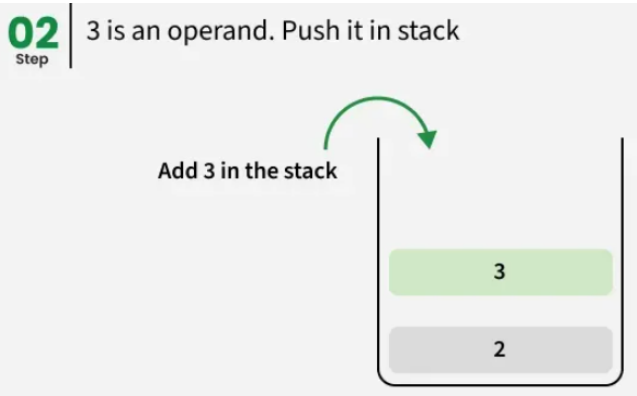
Enter the expression: 245+*

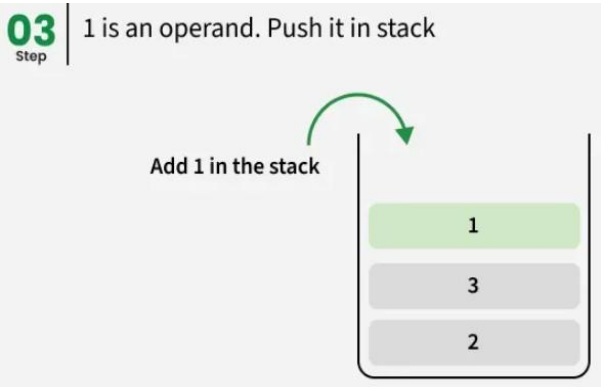
The result of expression 245+* = 18

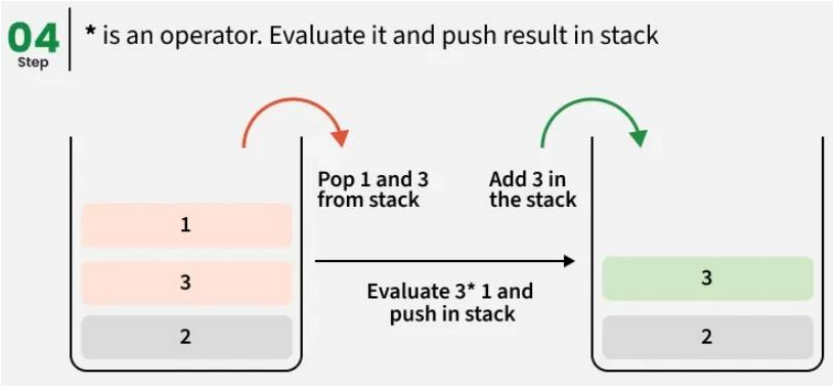
Expression in infix form is: 2 + (3 * 1) - 9. Now, evaluate step by step:

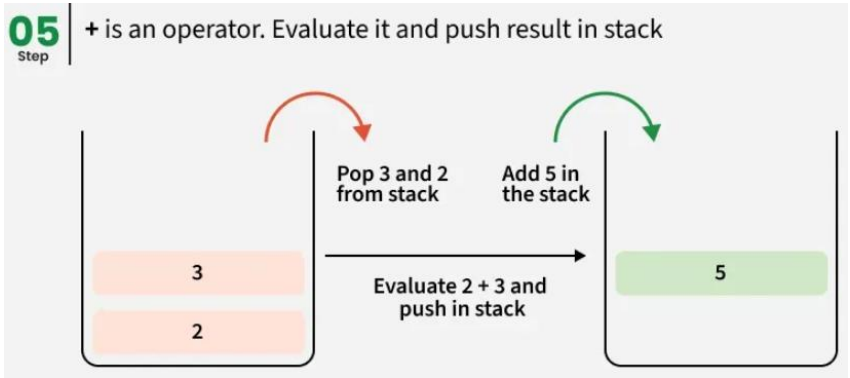
Postfix: 231*+9- = - 4

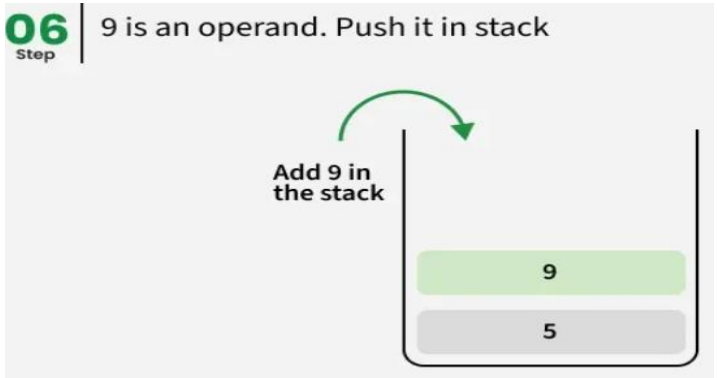


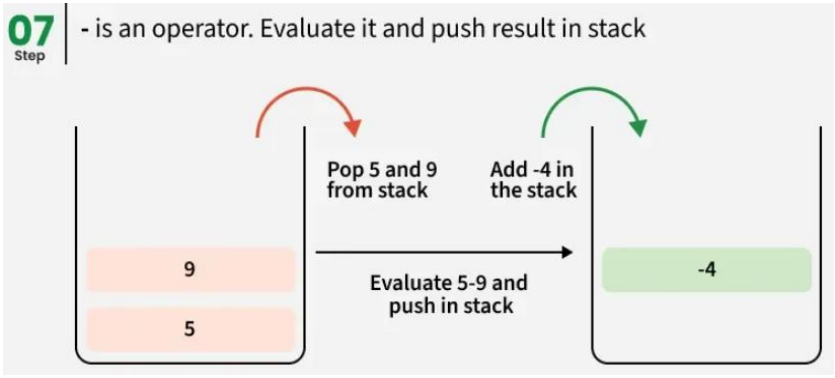


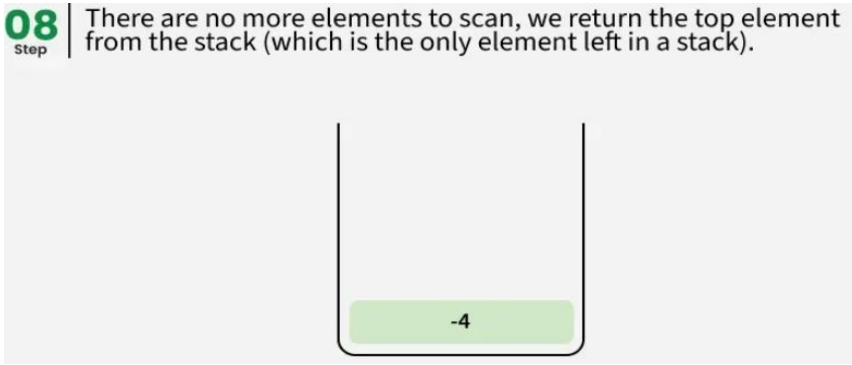












ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Check for balanced parentheses

```c
#include<stdio.h>
#include<string.h>
#include<stdlib.h>

#define MAX 30
int top=-1;
int stack[MAX];

void push(char);
char pop();
int match(char a,char b);
int check(char []);

int main()
{
        char exp[MAX];
        int valid;
        printf("Enter an algebraic expression : ");
        gets(exp);
        valid=check(exp);
        if(valid==1)
                printf("Valid expression\n");
        else
                printf("Invalid expression\n");

                return 0;

}
```

```c
int check(char exp[] )
{
        int i;
        char temp;
        for(i=0;i<strlen(exp);i++)
        {
                if(exp[i]=='(' || exp[i]=='{' || exp[i]=='[')
                        push(exp[i]);
                if(exp[i]==')' || exp[i]=='}' || exp[i]==']')
                        if(top==-1)    /*stack empty*/
                        {
                                printf("Right parentheses are more than left parenthese
                                return 0;
                        }
                        else
                        {
                                temp=pop();
                                if(!match(temp, exp[i]))
                                {
                                        printf("Mismatched parentheses are : ");
                                        printf("%c and %c\n",temp,exp[i]);
                                        return 0;
                                }
                        }
        }
        if(top==-1) /*stack empty*/
        {
                printf("Balanced Parentheses\n");
                return 1;
        }
        else
        {
                printf("Left parentheses more than right parentheses\n");
                return 0;
        }
}/*End of main()*/
```

```c
int match(char a,char b)
{
        if(a=='[' && b==']')
                return 1;
        if(a=='{' && b=='}')
                return 1;
        if(a=='(' && b==')')
                return 1;
        return 0;
}/*End of match()*/

void push(char item)
{
        if(top==(MAX-1))
        {
                printf("Stack Overflow\n");
                return;
        }
        top=top+1;
        stack[top]=item;
}/*End of push()*/

char pop()
{
        if(top==-1)
        {
                printf("Stack Underflow\n");
                exit(1);
        }
        return(stack[top--]);
}/*End of pop()*/
```

1. {[()]}
2. {[(])}
3. [{()}]
4. [()()]{}
5. ([]
6. ([{]})

Recursion

- The process in which a **function calls itself directly or indirectly is called recursion** and the **corresponding function is called as recursive function.**
- Using recursive algorithm, certain problems can be solved quite easily.
- Examples of such problems are **Towers of Hanoi (TOH), Inorder/Preorder/Postorder Tree Traversals, DFS of Graph,** etc.
- **Applications of Recursion**
- Recursion is widely used to solve different kinds of problems from simple ones like printing linked lists to being extensively used in AI. Some of the common uses of recursion are:
  - ✓ Tree-Graph Algorithms
  - ✓ Mathematical Problems
  - ✓ Divide and Conquer
  - ✓ Dynamic Programming
  - ✓ In Postfix to Infix Conversion
  - ✓ Searching and Sorting Algorithms

Types of Recursion

- Direct recursion
- Indirect recursion
- Tail recursion
- Head recursion

There is no statement, no operation before the function call.

In the below example, within the if block the first statement is the recursive function call. Before the recursive call, there is no statement present which means it is not performing any operation at the calling time. Further, after the recursive function call, the printf statement is there which is going to be executed at returning time. And hence this is an example of

Head Recursion.

```
#include <stdio.h>
void fun(int n)
{
if(n > 0)
{
fun(n-1);
printf ("%d", n);
}
}
int main()
{
fun(3);
return 0;
}
```

Output: 1 2 3

Tail recursion is defined as a recursive function in which the recursive call is the last statement that is executed by the function. So basically nothing is left to execute after the recursion call.

```
void print(int n)

{

if (n < 0)

    return;

     printf("%d ", n);

     print(n - 1); // The last executed statement is recursive call
}
```

Output: 3 2 1 0

Step	Function Call	Condition (`n > 0`)	Action Taken	Status of `printf`
1	`fun(3)`	3 > 0 (True)	Calls `fun(2)`	Paused (will print `3` later)
2	`fun(2)`	2 > 0 (True)	Calls `fun(1)`	Paused (will print `2` later)
3	`fun(1)`	1 > 0 (True)	Calls `fun(0)`	Paused (will print `1` later)
4	`fun(0)`	0 > 0 (False)	Returns immediately	Base case reached

```c
#include <stdio.h>

void rec(int n)
{   // Base Case
   if (n == 6) return;
   printf("Recursion Level %d\n", n);
   rec(n + 1);
}

int main()
{
   rec (1);
   return 0;
}
```

Output

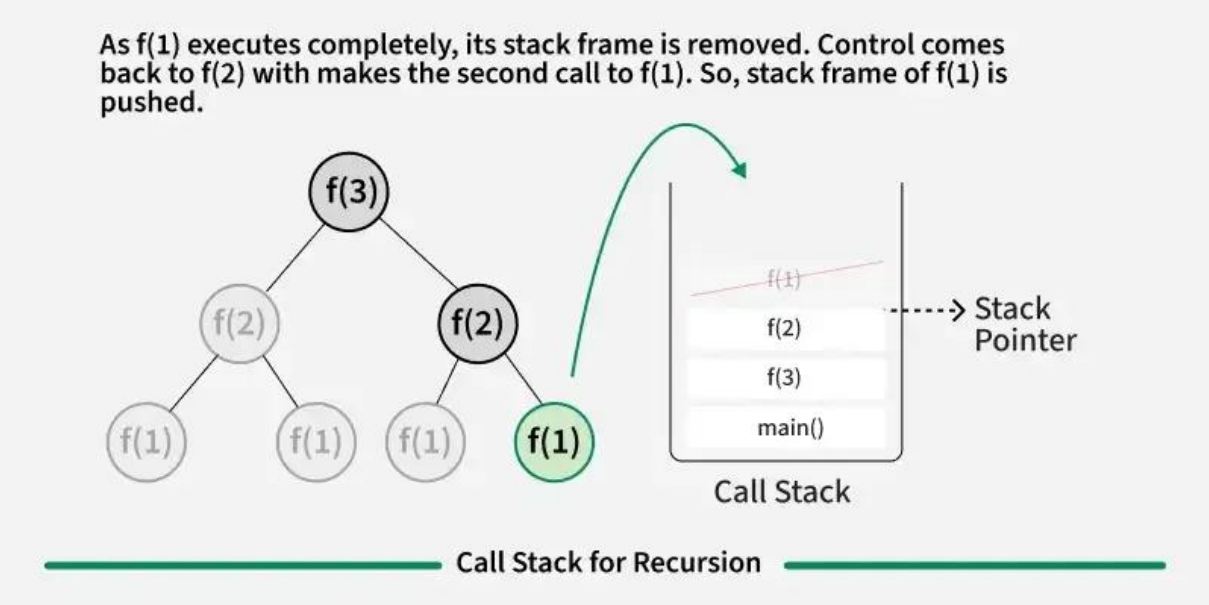
```
Recursion Level 1
Recursion Level 2
Recursion Level 3
Recursion Level 4
Recursion Level 5
```

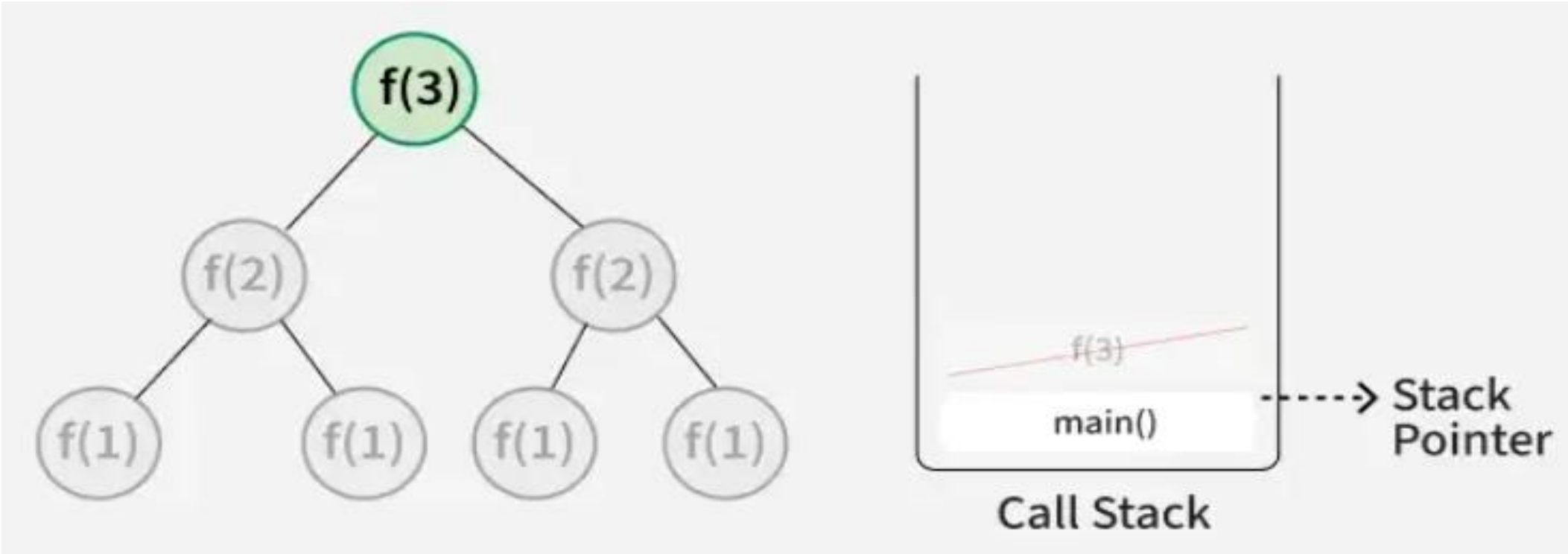
```c
#include <stdio.h>

void f(int n)
{
   printf("F(%d)'s Stack Frame Pushed\n", n);
    if (n > 1)
    {
     f(n - 1);
     f(n - 1);
    }
   printf("F(%d)'s Stack Frame Removed\n", n);
}

int main() {
   f(3);
   return 0;
}
```

```
F(3)'s Stack Frame Pushed
F(2)'s Stack Frame Pushed
F(1)'s Stack Frame Pushed
F(1)'s Stack Frame Removed
F(1)'s Stack Frame Pushed
F(1)'s Stack Frame Removed
F(2)'s Stack Frame Removed
F(2)'s Stack Frame Pushed
F(1)'s Stack Frame Pushed
F(1)'s Stack Frame Removed
F(1)'s Stack Frame Pushed
F(1)'s Stack Frame Removed
F(2)'s Stack Frame Removed
F(3)'s Stack Frame Removed
```





What is the Call Stack?

The call stack is a data structure used by the **program during runtime to manage function calls and local variables**. It operates in a Last-In-First-Out (LIFO) manner, meaning the last function called is the first one to complete and exit.

```
void printFun(int test)
{
  if (test < 1)
    return;
  else {
    printf("%d",test)
    printFun(test - 1); // statement 2
    printf("%d",test)
    return;
  }
}

int main()
{
  int test = 3;
  printFun(test);
}
```

output 3 2 1 1 2 3

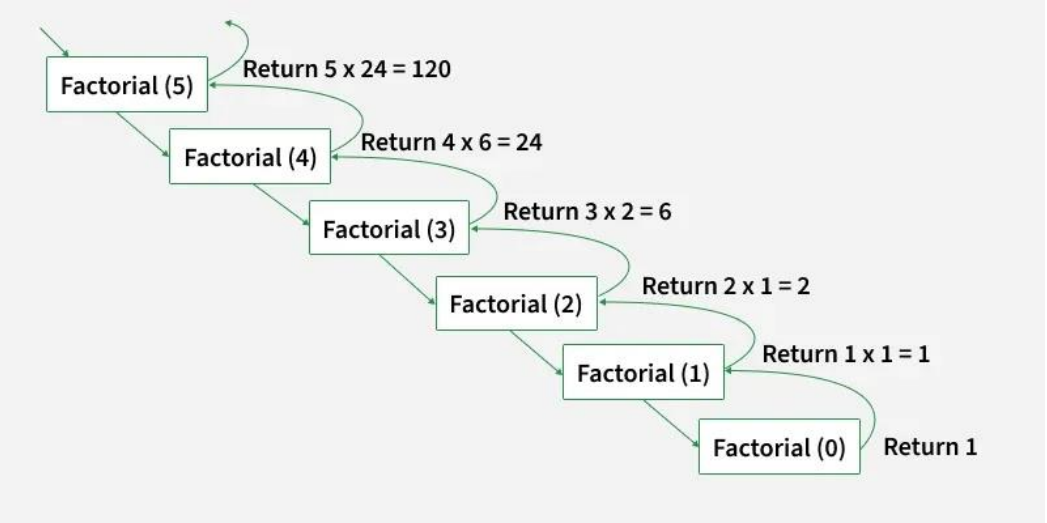
Example: Find a Factorial of n

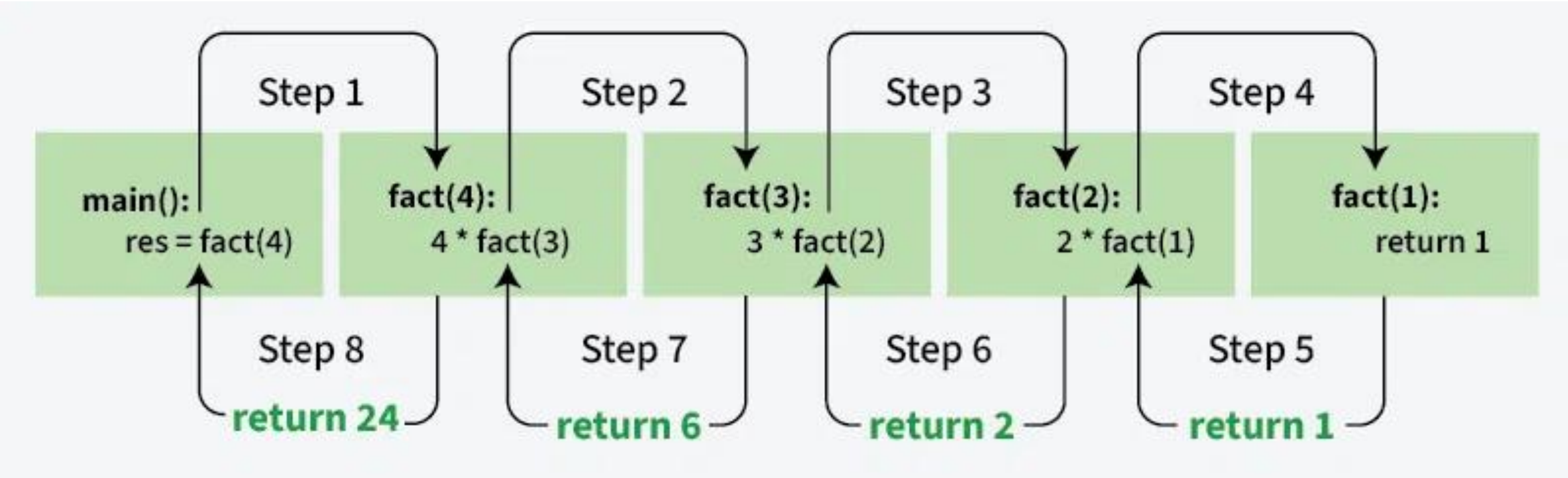
- factorial of n (n!) = 1 * 2 * 3 * 4 *... * n

```
#include <stdio.h>

int factorial(int n)
{
 if (n == 0) // Base Condition
  return 1;
 return n * factorial(n - 1);
}

int main()
{
   int num;
   printf("Enter a Positive number\n");
   scanf("%d", &num);
   printf("%d\n", factorial(num));
   return 0;
}
```




Step 1
Step 2
Step 3
Step 4
main():
res = fact(4)
fact(4):
4 * fact(3)
fact(3):
3 * fact(2)
fact(2):
2 * fact(1)
fact(1):
return 1
Step 8
Step 7
Step 6
Step 5
return 24
return 6
return 2
return 1

Example: Fibonacci series

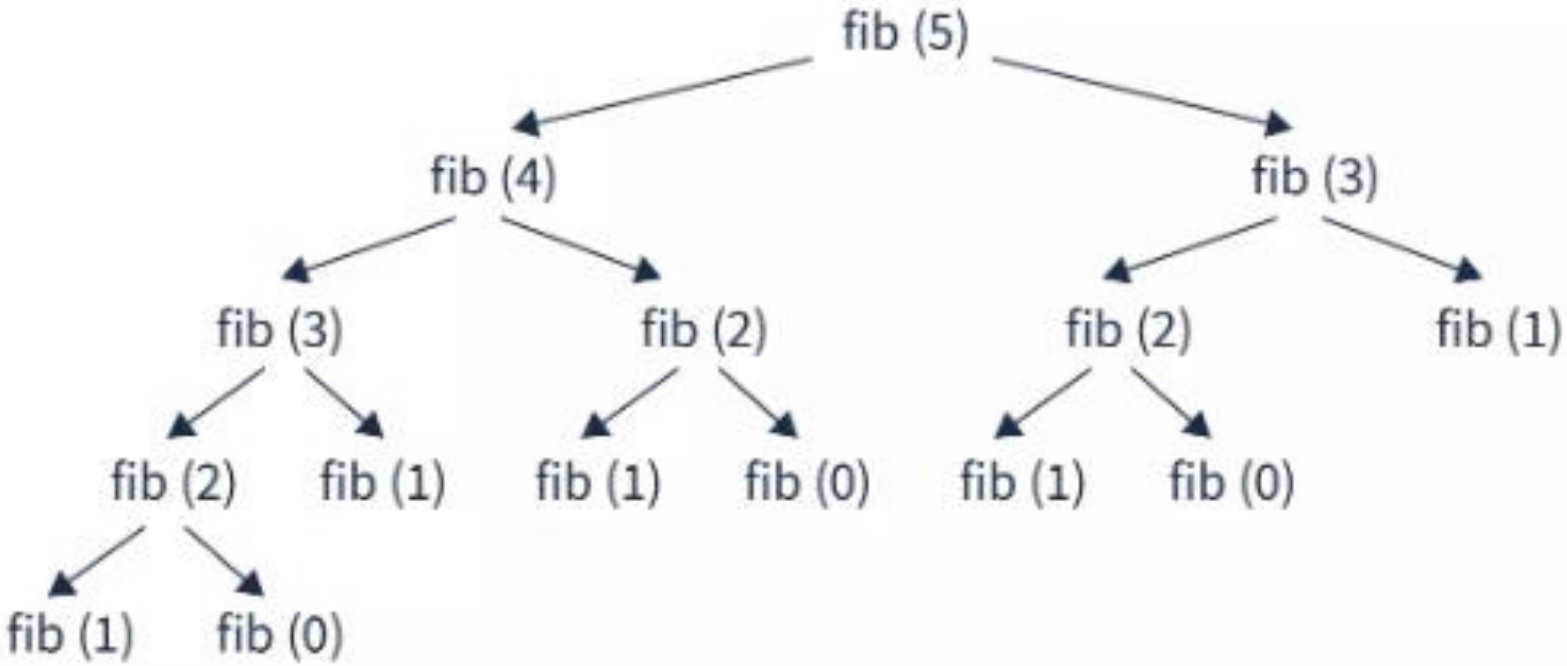
0 1 1 2 3 5 8 13 21 …. Fib(n) = Fib(n-1) + Fib(n-2)

```
//Fibonacci Series using Recursion
#include<stdio.h>
int fib(int n)
{
   if (n == 0) return 0;
   if (n == 1) return 1;
   return fib(n-1) + fib(n-2);
}

int main ()
{
 int n = 5;
 printf("%d", fib(n));
 return 0;
}
```

Recurrence relation:

- **Base case:** F(n) = n, when n = 0 or n = 1
- **Recursive case:** F(n) = F(n-1) + F(n-2) for n>1



Example: Indirect Recursion

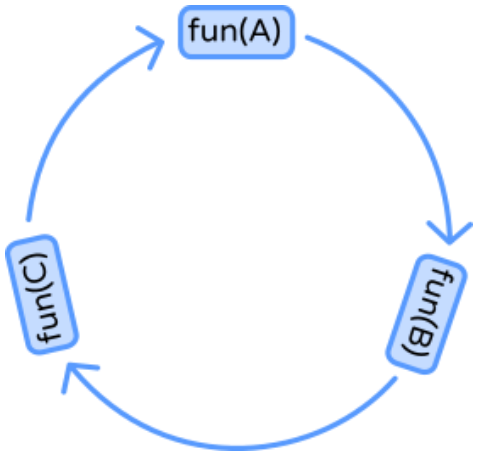
```
#include <stdio.h>
void functionB(int n);

void functionA(int n) {
if (n > 0) {
 printf("%d ", n);
 functionB(n - 1); // Indirect recursive call
 }
}

void functionB(int n) {
 if (n > 0) {
 printf("%d ", n);
 functionA(n / 2); // Indirect recursive call
 }
}
int main()
 {
 int num = 5;
 printf("Indirect Recursion: ");
 functionA(num);
 return 0;
}
```

Indirect Recursion: 5 4 2 1

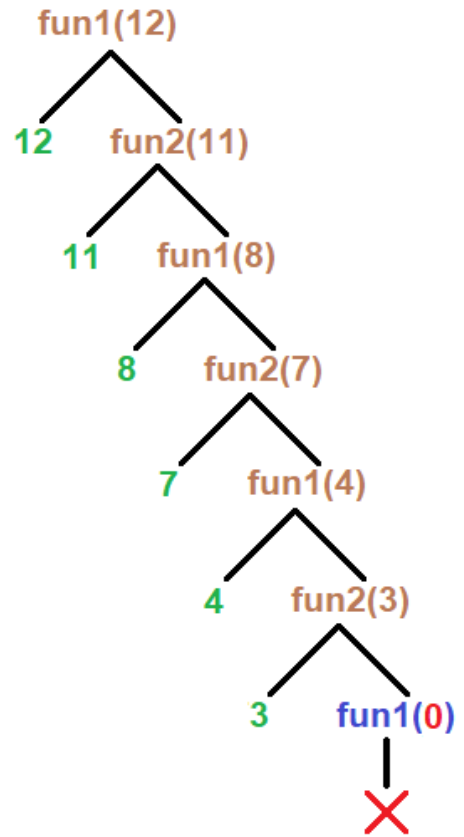
Indirect Recursion: In this recursion, there may be more than one functions and they are calling one another in a circular manner.




```
#include <stdio.h>
void fun1(int a)
{
if (a > 0)
{
printf("%d\n", a);
fun2(a - 1);
}
}

void fun2(int b)
{
if(b > 0)
{
printf("%d\n", b);
fun1(b - 3);
}
}

int main ()
{
int v = 12;
fun1 (v);
}
```



Output: **12 11 8 7 4 3**

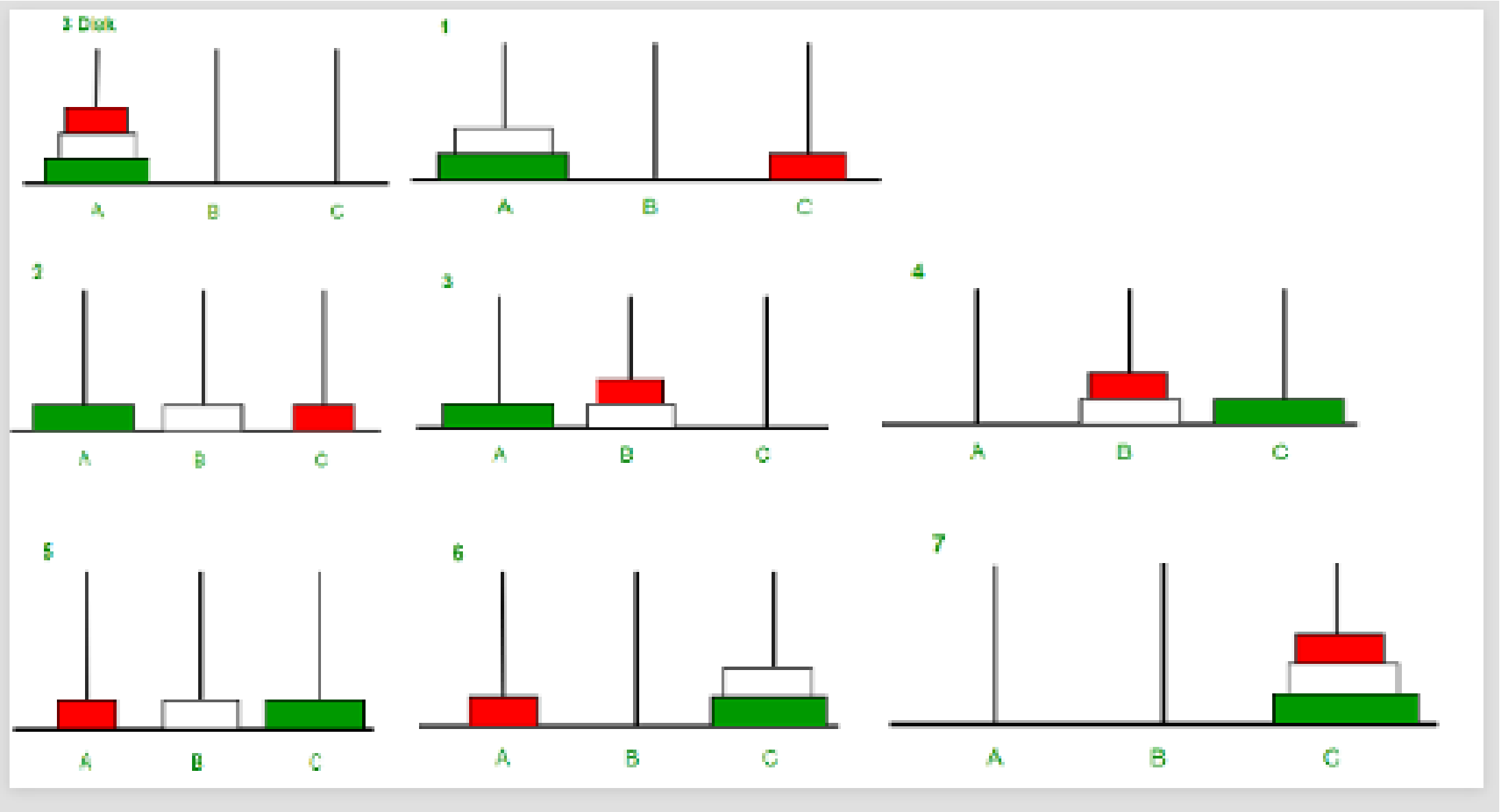
- Print odd number to n+1 and even number to n-1
- if n=10, print: **2,1,4,3,6,5,8,7,10,9**
- **Intial value n=1**

```
func odd(int n)
{
 if (n<=10)
  {
   printf("%d",n+1);
   n++;
   even(n);
  }
return;
}
```

```
func even(int n)
{
 if (n<=10)
  {
   printf("%d",n-1);
   n++;
   odd(n);
  }
return;
}
```

Current Call	Condition (`n <= 10`)	Operation & Printed Value	Value after `n++`	Next Function Called
`odd(1)`	True (1 ≤ 10)	Prints `n + 1` → `2`	`2`	`even(2)`
`even(2)`	True (2 ≤ 10)	Prints `n - 1` → `1`	`3`	`odd(3)`
`odd(3)`	True (3 ≤ 10)	Prints `n + 1` → `4`	`4`	`even(4)`
`even(4)`	True (4 ≤ 10)	Prints `n - 1` → `3`	`5`	`odd(5)`
`odd(5)`	True (5 ≤ 10)	Prints `n + 1` → `6`	`6`	`even(6)`
`even(6)`	True (6 ≤ 10)	Prints `n - 1` → `5`	`7`	`odd(7)`
`odd(7)`	True (7 ≤ 10)	Prints `n + 1` → `8`	`8`	`even(8)`
`even(8)`	True (8 ≤ 10)	Prints `n - 1` → `7`	`9`	`odd(9)`
`odd(9)`	True (9 ≤ 10)	Prints `n + 1` → `10`	`10`	`even(10)`

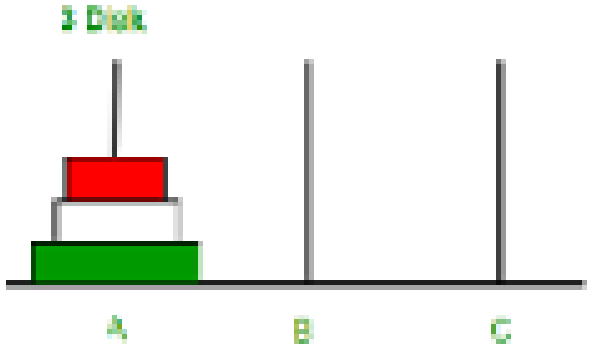
Example: Tower of Hanoi

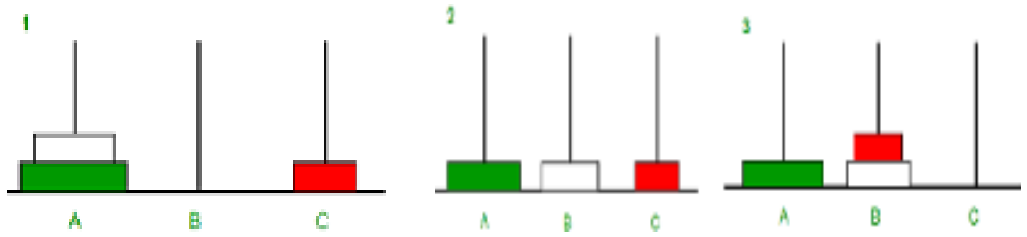


S1: move(n-1) disks from S to A using D

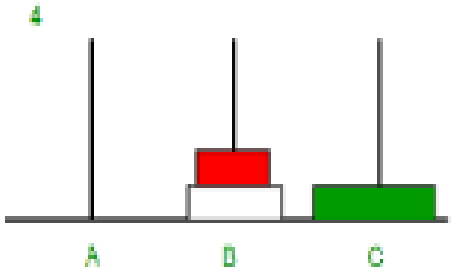
S2: move disk from S to D

S3: move(n-1) disks from A to D using S

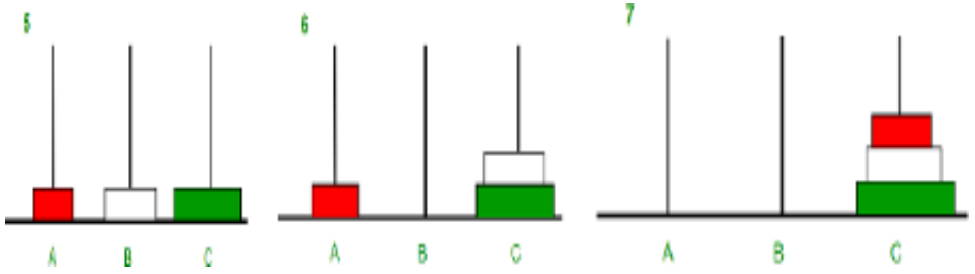




<-S1



<-S2



<-S3

```
Towerofhanoi(n,s,a,d)
{
    if(n==1)
  { print  "S ->D"
    return;
   }
  Towerofhanoi(n-1,s,d,a)
      printf "S->D"
  Towerofhanoi(n-1,a,s,d)
}
```

```c
/*
 * C program for Tower of Hanoi using Recursion
 */
#include <stdio.h>

void towers(int, char, char, char);

int main()
{
    int num;

    printf("Enter the number of disks : ");
    scanf("%d", &num);
    printf("The sequence of moves involved in the Tower of Hanoi are :\n");
    towers(num, 'A', 'C', 'B');
    return 0;
}
void towers(int num, char frompeg, char topeg, char auxpeg)
{
    if (num == 1)
    {
        printf("\n Move disk 1 from peg %c to peg %c", frompeg, topeg);
        return;
    }
    towers(num - 1, frompeg, auxpeg, topeg);
    printf("\n Move disk %d from peg %c to peg %c", num, frompeg, topeg);
    towers(num - 1, auxpeg, topeg, frompeg);
}
```

Trace the code

```
int sum(int n)
{
 if (n ==1)
   return 1;
   else
   return n+ sum(n-1);
}
```

n=4

10

Guess the output?

```
int f(int k, int n)
{
  if(n==k)
  return k; else
  if(n>k)
      return f(k, n-k);
    else
     return f(k-n, n)
}
```

f(6,8)

2

How many times get() will be called

```
void get(int n)
{
  if(n<1) return;
  get(n-1);
  get(n-3);
  printf("%d",n);
}
```

n=3

1 2 3

1. Consider the following recursive C function. If get(6) function is being called in main() then how many times will the get() function be invoked before returning to the main()?

```
void get (int n)
{
if (n < 1) return;
get(n-1);
get(n-3);
printf("%d", n);
}
```

(a) 15
(b) 25
(c) 35
(d) 45

Solution: Option (b)

2. Consider the following recursive C function that takes two arguments:

```
unsigned int foo(unsigned int n, unsigned int r) {
 if (n > 0) return (n%r + foo (n/r, r ));
else return 0;
}
```

What is the return value of the function foo when it is called as foo(345, 10)?

(a) 345
(b) 12
(c) 5
(d) 3

Solution: Option (b)

Explanation:
The call foo(345, 10) returns sum of decimal digits (because r is 10) in the number n. Sum of digits for 345 is 3 + 4 + 5 = 12.